

EXPERIMENT 2- File Copy Using System Calls

nano exp2.c

code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    source = open("source.txt", O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt",
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
```

```

    if (write(destination, buffer, bytesRead) != bytesRead)
    {
        perror("Error writing file");
        close(source);
        close(destination);
        return 1;
    }
}

if (bytesRead < 0)
{
    perror("Error reading file");
}

close(source);
close(destination);

printf("File copied successfully.\n");

return 0;
}

```

echo "Hello, this is my OSSP system call experiment." > source.txt

cat source.txt

output:

cat source.txt

File copied successfully.//output

```

yaswanth6970@Yash6970:~$ nano exp2.c
yaswanth6970@Yash6970:~$
yaswanth6970@Yash6970:~$ echo "Hello, this is a test file." > source.txt
yaswanth6970@Yash6970:~$ gcc exp2.c -o exp2
yaswanth6970@Yash6970:~$ ./exp2
File copied successfully.
yaswanth6970@Yash6970:~$ cat destination.txt
Hello, this is a test file.
yaswanth6970@Yash6970:~$ |

```

What are we doing?

We create a C program that copies one file to another.

Example:

source.txt



destination.txt

We use four system calls:

open()

read()

write()

close()

open()

Opens the file.

open → Open file

read()

Reads the contents.

File → read → Program

write()

Writes the contents into another file.

Program → write → New file

close()

Closes the file.

close → File closed

Why do we do this?

To understand how a program communicates with the Linux operating system to access files

Your practical also asks you to use strace to see the system calls made by cat sample.txt

OSSP Practical Experiments

What is strace?

strace shows the system calls happening behind a command

For example:

Bash

strace **cat** sample.txt

You may see:

openat()

read()

write()

close()

Remember:

open = open

read = read data

write = write data

close = close